

PLLM: Offline-Correlated Private Inference for Public-Weight Language Models

Blair Hudson · deployscience labs · blair@deployscience.com

11 September 2026

Abstract

PLLM is a high-performance multi-party inference runtime and autonomous research harness for private language-model systems. Its goal is to separate useful AI computation from access to prompts, context, state, and generated output. That separation could let independent, regional, sovereign, and energy-aware compute serve sensitive workloads without requiring one provider to hold the complete exchange. We investigate offline-correlated masking as one practical runtime design and implement it in PLLM for public-weight transformer models. A client creates one-time masks through a trusted Preparation service, which pushes $Wr - s$ corrections to an untrusted Inference service before a response. Online, the client sends a ticket and $x - r$; Inference returns $Wx - s$; and the client reconstructs Wx . The runtime binds prepared inventory to immutable model and stage commitments, atomically consumes tickets, burns unused reservations, packs prefill, and uses a persistent decode connection. Nine warm Qwen2.5-0.5B loopback runs across exact 30-, 63-, and 255-token contexts produced median time-to-first-token of 0.978, 1.374, and 5.158 seconds. Every run recorded zero online Preparation protocol work and zero plaintext prompt or token bytes in its audit counters. These results demonstrate that offline preparation can leave online inference to protected client-provider exchange while executing a complete public-weight language model.

Introduction

Hosted inference usually gives one model operator both the computation and the user’s plaintext prompts, token identities, activations, and generated output. Contracts, access controls, and retention policies can limit later use, but they do not remove that access. This coupling limits which providers can handle sensitive workloads and makes broader compute markets harder to build.

PLLM combines a Rust-first multi-party inference runtime with a research harness for implementing and comparing private inference methods. The current public-weight path separates access to client language from linear computation. The client runs tokenization, boundary matrices, attention, nonlinear operations, model state, sampling, and decoding. Remote services evaluate quantized transformer-body projections over one-time masked inte-

ger tensors. The current path uses no homomorphic encryption online and sends no plaintext prompt or token bytes to either remote role.

Private neural inference systems commonly combine homomorphic encryption, secret sharing, and secure two-party computation (Huang et al. 2022; Chen et al. 2022; Hao et al. 2022). PLLM instead exploits public weights and separates correlation generation in time. The resulting boundary is intentionally narrower: remote linear algebra is private under non-collusion, while nonlinear state remains at the client. This design has three concrete contributions:

1. an offline-correlation protocol with immutable inventory commitments, acknowledged readiness, one-use tickets, and fail-closed burn semantics;
2. a Python/Rust implementation with compact exact rings, packed prefill, persistent decode transport, local token boundaries, and authenticated role separation; and
3. a retained current-runtime evaluation that measures preparation, online latency, client traffic, lifecycle events, and privacy telemetry separately.

Protocol and threat model

Roles and arithmetic

The **Client** owns plaintext input, one-time root seeds, private activation scales, nonlinear state, boundary matrices, sampling, and output decoding. **Preparation** holds the public transformer body, expands masks, computes offline correlations, and is trusted to erase them. **Inference** holds the same public body and consumes correlations during online projection. Preparation and Inference are honest-but-curious and must not collude.

For a public quantized matrix W and private integer activation x , the Client and Preparation expand a fresh 32-byte root seed with SHAKE-256 into pseudorandom input mask r , output mask s , and ticket. Preparation computes

$$c = Wr - s.$$

It pushes c to Inference before the response. Online, the Client sends a random ticket and $x - r$. Inference atomically consumes the corresponding correction and returns

$$W(x - r) + c = W(x - r) + (Wr - s) = Wx - s.$$

The Client adds s and center-decodes the result. Assuming SHAKE-256 output is computationally indistinguishable from random, $x - r$ hides x from Inference and s hides the correction. Preparation knows r and s but does not receive $x - r$. Reuse would expose relationships between activations, so every row is consumed exactly once.

Offline inventory lifecycle

The Client first asks Inference to create an inventory committed to the model ID, immutable body fingerprint, complete stage set, quantization parameters, ring, wire width, and attempt budget. It authorizes this inventory through Preparation, then sends one root seed and row count per remote stage. Domain-separated expansion binds each derived r , s , and ticket to all commitments.

Preparation batch-computes $Wr - s$, pushes complete stage batches over a fixed authenticated WebSocket, and waits for accepted acknowledgement. Inference reports **READY** only after every committed stage has loaded. Online requests never invoke Preparation. A response reserves a disjoint row range atomically; cancellation, failure, replay, or early completion burns its unused tail. Inventories live in bounded process memory and disappear on replacement, restart, or idle expiry.

Security boundary

Inference does not receive literal plaintext prompts, token IDs, decoded output, seeds, masks, activation scales, local attention state, or sampling choices. Preparation does not receive online masked activations. Under pseudorandom masks, protocol-conforming services, mask non-retention, authenticated channels, and non-collusion, neither service receives the values needed to reconstruct client language alone. The data plane therefore removes the direct plaintext feed that a provider could otherwise retain for training.

This boundary does not cover all information. Both services observe the public model, stage names, tensor shapes, quantization, scheduling, timing, traffic volume, and approximate sequence length. Collusion or retained masks defeats the split. Authentication and TLS do not prove role independence, erasure, or correct execution, and the runtime does not defend against arbitrary malicious participants. Python releases mask objects after use, but physical memory zeroization has not been established. The client endpoint and local gateway remain inside the trusted boundary.

Implementation

PLLM is one Python distribution with a PyO3 extension backed by `p11m-core`. Rust owns validated immutable integer matrices, bounded coefficient arithmetic, codecs,

operating-system randomness, runtime AVX2 selection, and a persistent Rayon pool. Python owns the model graph, importers, quantization metadata, inventory protocol, scheduling, transport, and Response API.

Transformer matrices are quantized once and copied into Rust once at compilation. Each stage selects the smallest exact 2^{16} , 2^{24} , or 2^{32} ring from its signed output bound. Public bundles store each quantized matrix once and refer to it from stages. Tied embeddings share one row-quantized vocabulary matrix. Token lookup and output-head evaluation remain local, so vocabulary-sized projections do not cross the online boundary.

Prefill sends one ticket vector and one packed masked matrix per stage rather than one envelope and task per prompt row. Inference consumes the matching correction batch atomically. Decode uses one persistent authenticated WebSocket and a synchronous one-row correction path. Grouped-query attention reuses KV material; KV buffers grow geometrically; and an unused final-token transformer pass is deferred. These choices preserve one-use semantics while reducing framing, allocation, and event-loop work.

Three distinct credentials authenticate Client-to-Inference, Client-to-Preparation, and Preparation-to-Inference channels. Stage and model commitments prevent cross-session substitution. Request size, stage count, inventory bytes, reservations, and idle lifetime are bounded. The dashboard acts as a loopback OTLP collector: it archives immutable schema-3 summaries but never prompts, generated text, token IDs, activations, seeds, masks, credentials, or protocol payloads.

Evaluation

Method

The retained study used PLLM revision 277d19f, Python 3.13.15, and Qwen2.5-0.5B-Instruct revision 7ae557604adf67be50417f59c2c2f167def9a775 (Qwen Team 2024) on macOS 26.5.2 with an Apple M5 and 32 GiB memory. Client, Preparation, and Inference were separate loopback processes. One excluded warmup preceded nine warm runs: three repetitions at each exact 30-, 63-, and 255-token context. Output was capped at 16 tokens. Dashboard records were accepted only after an authoritative `response.completed` event and settled telemetry. The accompanying evidence artifact retains the public prompts, revision, model fingerprints, raw values, and limitations.

Latency. Table 1 reports medians; parentheses give the observed TTFT range. Online time starts after inventory readiness, while full time includes preparation and transition overhead. The shortest workload stopped after nine output tokens; the other workloads produced 16, so generation throughput is not compared between rows.

Traffic and preparation. Client traffic grows with activation rows and stage width. Correction bytes move from Preparation to Inference before online timing. The

Table 1: Current loopback latency; three runs per row.

Input	Output	TTFT (range), s	Online, s	Full, s
30	9	0.978 (0.935–0.982)	2.618	4.674
63	16	1.374 (1.353–1.379)	3.194	5.347
255	16	5.158 (4.961–5.289)	7.442	13.059

stage-row count is total matrix work across all 96 remote stages. The retained aggregate does not claim a per-run burn count.

All nine records shared the retained model and body fingerprints, recorded zero online Preparation requests and operations, and recorded zero plaintext prompt and token bytes in the audit counters. The installed-wheel transport smoke completed all three roles with nonzero process metrics and a one-signal clean shutdown. The repository gate passed 400 Python tests with native kernels, the same 400 with forced scalar kernels, and 11 Rust tests.

These measurements isolate current implementation behavior, not general model performance. They use one CPU host and loopback networking; include no GPU, WAN, concurrency, malicious-provider test, or output-quality evaluation. Preparation is offline with respect to token latency but remains real compute and traffic. At 255 input tokens it dominates the difference between online and full-response time, while client traffic reaches 432.71 MB. Private execution therefore removes provider access to client data, not communication cost.

Conclusion

PLLM demonstrates public-weight language-model inference in which remote providers perform useful matrix work without receiving plaintext client language. Offline one-time correlations remove Preparation from the online path; immutable commitments, atomic consumption, and burn semantics close the inventory lifecycle; and packed transport makes real Qwen execution practical on a CPU loopback. The measured result is a precise boundary: private content under non-colluding, honest-but-curious roles, with visible metadata and substantial client traffic.

References

- Chen, Tianyu, Hangbo Bao, Shaohan Huang, et al. 2022. “THE-X: Privacy-Preserving Transformer Inference with Homomorphic Encryption.” *Findings of the Association for Computational Linguistics: ACL 2022*, 3510–20. <https://doi.org/10.18653/v1/2022.findings-acl.277>.
- Hao, Meng, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. 2022. “Iron: Private Inference on Transformers.” *Advances in Neural Information Processing Systems 35*, 15718–31. <https://doi.org/10.52202/068431-1143>.
- Huang, Zhicong, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. 2022. “Cheetah: Lean and Fast Secure Two-Party Deep Neural Network Inference.” *31st USENIX Security Symposium*, 809–26. <https://www.usenix.org/conference/usenixsecurity22/presentation/huang-zhicong>.
- Qwen Team. 2024. *Qwen2.5 Technical Report*. arXiv:2412.15115. <https://doi.org/10.48550/arXiv.2412.15115>.

Table 2: Lifecycle traffic and offline preparation.

Input	Client I/O	Correction push	Stage rows
30	63.33 MB	59.80 MB	6,144
63	126.29 MB	72.88 MB	7,488
255	432.71 MB	252.18 MB	25,920